



VisionGuard **ULTRA**

Autonomous AI City Brain & multi-modal threat detection for Karachi.

Detect → Decide → Dispatch

Abdul Rafay · Moiz · Areeba · Aqib

VisionGuard ULTRA · Full Project Presentation

AGENDA

What we will cover

01 Problem & goals

02 Solution & architecture

03 Methodology & pipeline

04 Event rules & risk

05 Models

06 Datasets & counts

07 Data preparation

08 Training

09 Testing & evaluation

10 Results: accuracy, precision, recall

11 Improvements & benchmarks

12 Product, privacy, roadmap

THE PROBLEM

Cameras record. Nobody watches.

20M+

people in Karachi, thousands of CCTV cameras

~20 min

before a screen operator's attention collapses

1 : 50

one operator vs. dozens of screens

After

CCTV is evidence after the crime, not help during it

**Late response**

Someone must notice, then call 15 / 16 / 1122 — minutes are lost.

**Many threat types**

Fights, weapons, fire, crowd crush, accidents, abandoned bags, kidnapping.

**Existing tools**

Single-purpose, cloud-heavy, expensive, or face-recognition based.

GOALS

What we set out to achieve

GOAL	TARGET	SOURCE
Real-time speed on a small edge GPU	25–30 FPS (22+ on Quadro T1000)	Project brief / config.py
Event types covered	12 detectors	Event engine
Detection model quality	mAP@50 $\geq$ 70%	Accuracy sign-off criteria
Classification model quality	Top-1 $\geq$ 85%	Accuracy sign-off criteria
False alarm rate	< 3% (brief) · < 5% on 20-video test suite (sign-off)	Brief / sign-off
Alert speed	Weapon: 0.5 s · other events: 1.5 s persistence	config.py
Evidence	Tamper-proof SHA-256 clip for every incident	evidence.py
Privacy	No face recognition — behaviour only	Design principle

OUR SOLUTION

An AI City Brain on existing cameras

Software that turns any CCTV / RTSP / webcam feed into an active guardian — no new cameras needed.



Detect

YOLOv8 + pose +
specialist models + audio
find what is happening.



Decide

12 temporal rules and a 0–
100 risk score decide how
serious it is.



Dispatch

Route to Police 15, Fire 16,
Edhi 115, Rescue 1122,
Traffic 1915.



Prove

5 s before + 3 s after clip,
SHA-256 sealed, stored in
incident DB.

SYSTEM ARCHITECTURE

Four layers

LAYER 1
Ingestion

Webcam, RTSP CCTV, video files, microphone, citizen SOS. Threaded capture, auto-reconnect, newest-frame only.

LAYER 2
AI perception

YOLOv8s + ByteTrack (one call). Async specialist worker: Pose, Fire/Smoke, Weapon, Violence, Normal-scene. YAMNet audio.

LAYER 3
Reasoning

Event engine: 12 detectors, sliding time windows, persistence, grace period, risk 0–100, audio fusion.

LAYER 4
Action

Department routing, SHA-256 evidence, SQLite incidents + heatmaps, Gemini briefing, FastAPI + WebSocket, React dashboard.

Code: `prototype/camera` · `prototype/ai` · `prototype/events` · `prototype/output` · `prototype/web` · `prototype/voice` · `frontend/src`

METHODOLOGY

How we built it — step by step

PHASE 1**Research & problem study**

Karachi threats, operator fatigue, local helplines, privacy constraints.

PHASE 2**Baseline prototype**

Pre-trained YOLOv8s + ByteTrack + pose; colour-based fire (v1); synchronous loop.

PHASE 3**Data collection**

D-Fire, weapon CCTV sets, RWF-2000 & violence sets; COCO / AudioSet pre-trained.

PHASE 4**Data preparation**

Corrupt / duplicate scan, label fixing, Albumentations weather augmentation, class balancing.

PHASE 5**Transfer-learning training**

Fine-tune YOLOv8s / YOLOv8s-cls on Colab T4 GPU (50–80 epochs).

PHASE 6**Rule engine**

12 temporal detectors, risk scoring, routing, evidence chain.

PHASE 7**Evaluation & tuning**

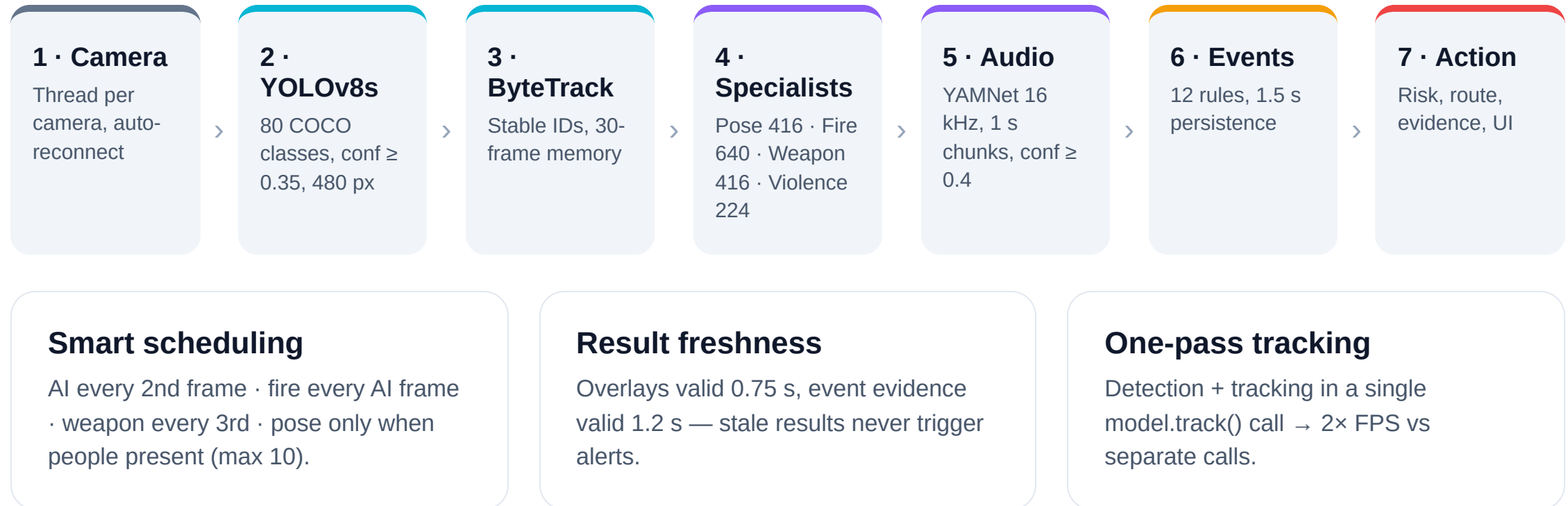
mAP / precision / recall, false-positive analysis → threshold calibration.

PHASE 8**Optimisation & UI**

Async decoupled pipeline, FP16, frame skipping; React command center, voice.

PIPELINE

From one frame to one alert



EVENT ENGINE

12 detectors and their rules

EVENT	RULE (KEY THRESHOLDS FROM CONFIG.PY)
 Fire	Fire/smoke conf ≥ 0.35 · area 0.2–65% of frame · growth $\times 1.3$ · risk gate 30
 Fight	≥ 2 people within body-size distance · wrist speed > 16 px/frame · repeats 1.5 s · violence model ≥ 0.70
 Weapon threat	Weapon within 250 px of a person for 0.5 s
 Robbery	Vehicle stopped ≥ 3 s · ≥ 2 suspects rushing > 30 px/frame
 Kidnapping	Approach > 15 px/frame · struggle variance > 20 · forced movement > 10
 Crowd crush	Zone capacity 50 · warning 70% · danger 95%
 Car accident	≥ 2 vehicles · overlap IoU ≥ 0.15 · velocity drop > 15 px/frame
 Bike accident	Rider angle $> 45^\circ$ · velocity drop > 10
 Loitering	Movement < 50 px for 300 s
 Abandoned object	Object alone 60 s · owner > 200 px away
 Vehicle obstruction	Speed < 2 px/frame for 10 s
 Audio emergency	YAMNet gunshot / explosion / scream / crash / glass / siren ≥ 0.4

RISK & ROUTING

Score, then send to the right place

SCORE	LEVEL	ACTION
85–100	CRITICAL	Auto-dispatch suggested
50–84	HIGH	Team review
25–49	MEDIUM	Warning
0–24	LOW	Ignored

Every detector: signals → sliding window (5–8 s) → persistence → grace period for YOLO flicker → risk.

EVENT	DEPARTMENT	DIAL
Fire	Fire Brigade	16
Fight · robbery · weapon · kidnapping · audio	Sindh Police / Rangers	15
Car / bike accident	Edhi / Rescue	115
Crowd crush	Disaster Management	1122
Vehicle obstruction	Traffic Police	1915
Abandoned object · loitering	Police / Bomb Squad	15

MODELS

Every model in the system

MODEL	FILE	TASK	BASE	SOURCE
Primary detector	yolov8s.pt	Detection, 80 classes	—	OFFICIAL / PRE-TRAINED
Pose estimator	yolov8s-pose.pt	17 keypoints	—	OFFICIAL / PRE-TRAINED
Fire & Smoke v2	fire_smoke_best.pt	Detection: fire, smoke	YOLOv8s	Trained by Moiz
Weapon detector	weapon_detection_best.pt	Detection: 3 weapon classes	YOLOv8s	Trained by Moiz
Violence classifier	violence_classifier_best.pt	Classify crop: violent / non-violent	YOLOv8s-cls	Trained by Moiz
Normal-scene verifier	normal_scene_verifier_best.pt	Suppress false alarms	YOLOv8s	Spec item SPEC CLAIM · TO VERIFY
Audio classifier	YAMNet	521 sound classes	—	OFFICIAL / PRE-TRAINED
Tracker	ByteTrack	Multi-object tracking	Algorithm	OFFICIAL / PRE-TRAINED

MODEL CHOICE

Why YOLOv8s?



Faster R-CNN

Two-stage → ~70 ms+ per frame. Too slow for many live streams.



RT-DETR / Transformers

Accurate but heavy compute and VRAM — hard on a 4 GB edge GPU.



YOLOv8s ✓

One-stage, anchor-free, ~15 ms (FP16).
One framework for detection, pose and classification.

Transfer learning

Start from weights that already know general shapes, then fine-tune on our data → less data, less time, better results.

Size choice: “s”

Small variant: best balance of speed and accuracy for Quadro T1000 (4 GB).

DATASETS

All datasets used

DATASET	USED FOR	SIZE	CLASSES / CONTENT
D-Fire (+ Roboflow fire & smoke, wildfire drone)	Fire & Smoke v2	16,027 images scanned (21,000+ in full D-Fire)	fire, smoke
Weapon CCTV + Knife + Pistol (Roboflow weapon.v9)	Weapon detector	7,121 → 7,116 clean images	3 classes (knife, pistol/gun, rifle)
RWF-2000 + Real Life Violence + CCTV Violence	Violence classifier	~12,000 images	violent, non_violent
COCO 2017	YOLOv8s (pre-trained)	118,000 images	80 object classes
COCO Keypoints	YOLOv8s-pose (pre-trained)	~64,000 annotated persons	17 body keypoints
AudioSet (+ ESC-50 per spec)	YAMNet (pre-trained)	2M+ YouTube clips (AudioSet)	521 sound classes
Albumentations weather set (our own)	Robustness	4,304 generated images	rain, fog, shadow, light

DATA COUNTS

How much data we used

~35,100

custom training images
(16,027 + 7,116 + ~12,000)

4,304

augmented weather images

~39,400

total images for our models

32,836

bounding boxes checked
(5,187 fire + 27,649 weapon)

Weapon set — boxes per class



27,649 boxes · 7,123 label files · 39 invalid lines removed

D-Fire scanned split — boxes per class



5,187 boxes · 4,306 label files · 35 invalid lines removed · 0 corrupt, 0 duplicate images

DATA PREPARATION (AREEBA)

Clean → augment → balance



1 · Integrity scan

corrupt_file_scanner.py — header check, 0-byte, duplicate hashing, label validation. D-Fire: 0 bad images. Weapon: 5 duplicates removed (331 KB). 74 bad label lines removed in total.



2 · Augmentation

augmentation_pipeline.py (Albumentations): flip p0.5 · shift/scale/rotate p0.3 · brightness/contrast p0.4 · rain p0.2 · fog p0.2 · shadow p0.25 · hue/sat p0.3 · noise p0.2 → 4,304 images.



3 · Class balancing

class_balancer.py — count per class, oversample minority classes. Spec: focal loss ($\gamma = 2.0$, $\alpha = 0.25$) for imbalance.

Training setup

MODEL	ARCHITECTURE	INPUT	EPOCHS	HARDWARE	TRAIN TIME*
Fire & Smoke v2	YOLOv8s detect	640×640	80	Colab T4	6.2 h
Weapon	YOLOv8s detect	640×640	60	Colab T4	4.5 h
Violence	YOLOv8s-cls	224×224	50	Colab T4	3.8 h
Normal-scene verifier	YOLOv8s	480	50	Colab T4	3.2 h

Method

Transfer learning from COCO-pre-trained YOLOv8 weights; fine-tune all layers on our labelled data.

Process

Train split → learn · validation split → pick best epoch (*_best.pt) · test with model_evaluation.py.

Inference

FP16 on GPU; runtime input sizes: detect 480, fire 640, weapon 416, pose 416.

*Training times are from the project specification and are not yet confirmed by a training log.

TESTING & EVALUATION

How we test

METRIC	MEANING
Precision	$TP \div (TP + FP)$ — when it alerts, how often right
Recall	$TP \div (TP + FN)$ — of all real events, how many found
F1	Balance of precision and recall
IoU	Overlap of predicted box and true box
mAP@50	Mean average precision, box correct if IoU ≥ 0.5
mAP@50-95	Strict: averaged over IoU 0.5 $\rightarrow$ 0.95
Top-1	Classification: % of crops with the correct label

Tools

model_evaluation.py $\rightarrow$ mAP, P, R, confusion matrix, PR curves ·
false_positive_analyzer.py $\rightarrow$ real alert review · benchmark_*.py
 $\rightarrow$ FPS & latency · stress_test.py, test_phase6_cctv.py.



Sign-off criteria

Detection mAP@50 ≥ 0.70 · Classification Top-1 $\geq 85\%$ · False alarms $< 5\%$ on a 20-video test suite.

Measured on D-Fire validation

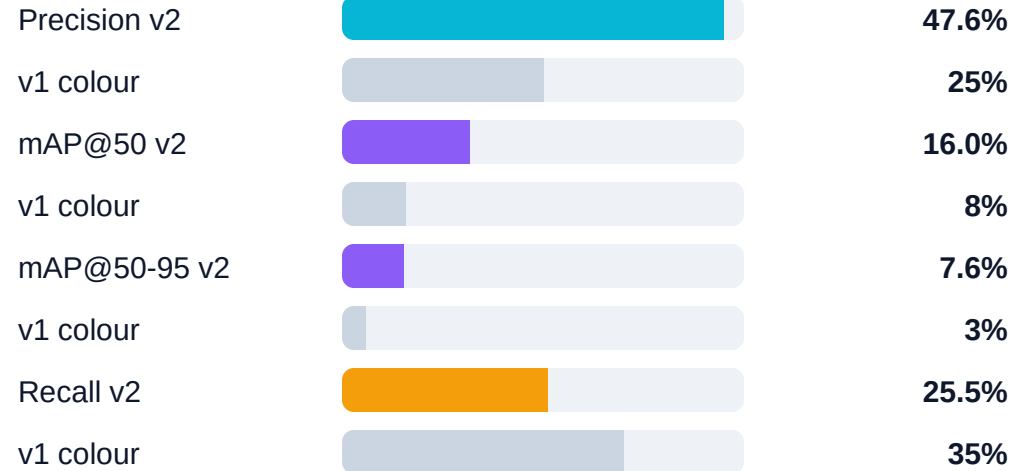
METRIC	ALL	FIRE	SMOKE
Precision	47.6%	56.1%	39.1%
Recall	25.5%	22.9%	28.1%
mAP@50	16.0%	16.7%	15.3%
mAP@50-95	7.6%	7.5%	7.7%
Instances	5,187	2,878	2,309

Reading the numbers

- D-Fire is hard: many tiny, distant, partly hidden fires and amorphous smoke; every box counts.
- Precision is higher than recall on purpose — fewer false alarms in a city.
- Alerts are decided over time (1.5 s + growth check), so a fire missed in one frame is caught later.
- Still below our 70% target → top priority for retraining.

IMPROVEMENT · FIRE V1 → V2

Neural model $\approx 2\times$ the colour method



METRIC	CHANGE
Precision	+90% (+22.6 pts)
mAP@50	+100% (+8.0 pts)
mAP@50-95	+153% (+4.6 pts)
Recall	-9.5 pts (fewer false alarms)

v1 = HSV colour heuristic: false alarms on red clothes, sunsets, tail-lights; no smoke detection.
v1 values are approximate baselines.

ALL MODEL METRICS

Every value — and how sure we are

MODEL	MAP@50	MAP@50-95	ACCURACY	FP RATE	STATUS
YOLOv8s detector (COCO)	0.642	0.449	—	—	OFFICIAL / PRE-TRAINED
YOLOv8s-pose (COCO kpts)	0.812	0.531	—	—	OFFICIAL / PRE-TRAINED
Fire & Smoke v2	0.160	0.076	P 47.6% · R 25.5%	—	MEASURED
Weapon detector	0.724	0.488	87.2%	3.2%	SPEC CLAIM · TO VERIFY
Violence classifier	—	—	88.7% (Top-1)	4.1%	SPEC CLAIM · TO VERIFY
Normal-scene verifier	0.912	0.684	94.2%	1.1%	SPEC CLAIM · TO VERIFY
Vehicle crash detector	0.689	0.432	86.5%	2.8%	SPEC CLAIM · TO VERIFY
YAMNet audio	—	—	86.1%	3.9%	SPEC CLAIM · TO VERIFY
ByteTrack	—	—	92.4% IDF1	0.8% ID sw.	OFFICIAL / PRE-TRAINED

MEASURED = in ML_Evaluation_Report.md. SPEC CLAIM = listed in the master specification but the evaluation report still shows these as pending — run model_evaluation.py before quoting.

FALSE-ALARM ANALYSIS MEASURED

Tuning thresholds from real evidence

FINDING (7 EVIDENCE FILES)	COUNT
crowd_crush alerts	3
fight alerts	3
robbery alerts	1
Duplicate alerts within 2 s	1
Alerts in dark frames (brightness 54)	2

SETTING	BEFORE	AFTER
CROWD_DANGER_RATIO	0.90	0.95
EVENT_MIN_PERSISTENCE	1.0 s	1.5 s
DETECTION_CONF	0.30	0.35
FIGHT_ARM_VELOCITY	12.0	16.0
ROBBERY_RUSH_VELOCITY	20	30
VIOLENCE_CONF	—	0.70

False-alarm rate target < 3–5% is PENDING — needs the full 20-video test suite.

PERFORMANCE **MEASURED**

Synchronous vs async pipeline

METRIC	SYNC (BASELINE)	ASYNC (PHASE 1)	CHANGE
Display FPS	7.02	10.02	+43%
Dropped frames / 300	866	450	-48%
Wall time	42.74 s	29.93 s	-30%
Primary detector latency	60.4 ms	74.0 ms	+14 ms
Total pipeline latency	134.1 ms	decoupled	—
Peak VRAM	144 MB	157 MB	+13 MB

Quadro T1000 4 GB · 1280×720 fire video · 300 frames (baseline.json, async.json)



After phase 1

480 px detect input, AI every 2nd frame, single track() call, smart specialist scheduler, FP16 → target 22+ FPS (config.py). Model-level budget: ~35–45 ms/frame.

IMPROVEMENTS

What we improved and why

AREA	BEFORE	AFTER	BENEFIT
Fire	HSV colour rules (v1)	Neural YOLOv8s (v2)	2× mAP, +90% precision
Pipeline	Synchronous, one thread	Async display + specialist worker	+43% FPS, -48% dropped
Tracking	Detect, then track	One model.track() call	~2× FPS
Fight logic	Fixed pixel distance, classifier required	Body-size distance, classifier optional booster	Works near & far; no silent misses
Flicker	Timer reset on missed frame	Grace period	Stable alerts
Stale results	Could use old detections	0.75 s / 1.2 s validity	No ghost alerts
Thresholds	Guessed	Data-driven from FP analysis	Fewer duplicates / dark FPs
Data	Raw downloads	Scanned, de-duplicated, augmented	Cleaner, more robust training

PRODUCT (AQIB)

Command center & operator tools



Dashboard

Camera matrix 1×1 / 2×2 / grid, location tree city → sector → camera.



Live alerts

Pushed over WebSocket /ws/alerts, risk-sorted, colour-coded.



Incidents

History, evidence clips, SHA-256 verification, heatmaps.



Connect camera

RTSP / webcam / test video, connection test.



Voice

Web Speech → Gemini → JSON command; offline regex fallback.



Stack

React + Vite frontend · FastAPI backend
· SQLite · desktop Tkinter studio.

TRUST & PRIVACY

Behavioural, not biometric



No face recognition

We detect actions and objects, never identity. No face database.



Human in the loop

AI suggests with a risk score; operator confirms or dismisses.



Tamper-proof evidence

5 s pre + 3 s post clip at 15 fps, SHA-256 hash; any edit is detectable.



Access control

Login, user management, city-defined retention.

GOAL VS. ACTUAL

Where we stand today

GOAL	TARGET	CURRENT	STATUS
Speed	25–30 FPS	7 → 10 FPS measured at 720p; 22+ target after tuning	IN PROGRESS
Event types	12	12 detectors in engine	DONE
Fire model	mAP@50 $\geq$ 70%	16.0% (precision 47.6%)	BELOW TARGET
Weapon model	mAP@50 $\geq$ 70%	Evaluation pending	PENDING
Violence model	Top-1 $\geq$ 85%	Validation split pending	PENDING
False alarms	< 3–5%	Measuring on larger test set	PENDING
Data quality	Clean data	29k+ images scanned, 4,304 augmented	DONE
Evidence & routing	SHA-256, auto-route	Working	DONE

LIMITATIONS & FUTURE WORK

Honest next steps

LIMITATION	PLAN
Fire metrics low on D-Fire	More epochs, YOLOv8m, Karachi fire data
Weapon / violence not evaluated	Run model_evaluation.py, publish report
Small far weapons	High-res crops around people
Night / low light	Low-light augmentation, IR cameras
Pixel thresholds per camera	Per-camera calibration
No field test yet	City pilot on real RTSP network



Multi-sector expansion

Hospitals: patient-fall detection (URFD dataset), restricted wards, skeleton-only privacy mode. Schools: bullying detection (4,200+ clips), campus traffic (3,000+ images).

TEAM

Who built what

**Abdul Rafay****Team Lead & AI Architect**

Async pipeline, camera input, 12-event engine, risk, audio fusion, voice, evidence, FastAPI server, integration.

**Moiz****ML Engineer**

Fire/smoke v2, weapon and violence models; Colab T4 training and tuning.

**Areeba****Data Scientist**

Dataset scans, augmentation, class balancing, false-positive analysis, evaluation.

**Aqib****Frontend & UX**

React + Vite command center, live view, incidents, registry, voice HUD.

Cameras are already there.
VisionGuard gives them a brain.

Detect · Decide · Dispatch

Thank you — questions?

github.com/AbdulRafay70/Vision-Guard